

# Parallel Morphological Image Processing

Accelerating morphological operations via C++17 and OpenMP

**Davide Lombardi**

July 10, 2026



UNIVERSITÀ  
DEGLI STUDI  
FIRENZE

Da un secolo, oltre.

# Introduction & Objectives



- **Mathematical Morphology (MM):** Robust framework for extracting region descriptors (boundaries, skeletons) and noise reduction.
- **The Goal:** Accelerate morphological operations for high-resolution imagery, transitioning from a sequential approach to a highly optimized parallel framework.
- **Technology Stack:** C++17 and OpenMP.
- **Benchmark Dataset:** BSDS500 (converted to grayscale, scaled up to 4x to evaluate scalability).

# Context & Sequential Baseline

○

## Fundamental Operators

- **Erosion:** Minimum local value (discards small-scale noise).
- **Dilation:** Maximum local value.

## The Bottleneck

- Naive 2D stencil execution scales quadratically:  $\mathcal{O}(M \times N \times K^2)$ .
- **Compound operations** (Opening/Closing) require multiple full-image passes.
- Destroys *temporal locality* and creates severe memory bandwidth issues.

# Hardware Approach: OpenMP Parallelization

## Exploiting Embarrassingly Parallel workloads

- Morphological stencils are *read-only* regarding the input state (no loop-carried dependencies).
- **Loop Collapsing:** `collapse(2)` merges X and Y spatial loops, increasing work granularity and preventing thread starvation.
- **Memory Safety:** Use of `default(none)` and strict variable scoping (e.g., `local min_val`) avoids locks/atomic synchronizations.
- **Scheduling Strategy:** While workload per pixel is theoretically static, **dynamic scheduling** marginally outperforms static by mitigating *false sharing* and OS scheduling jitter.

# Algorithmic Optimization: 1D Separable Kernels



- **The Dominant Lever:** Transitioning from a 2D stencil to two consecutive 1D passes (Horizontal  $\rightarrow$  Vertical).
- **Complexity Reduction:** From  $\mathcal{O}(K^2)$  down to  $\mathcal{O}(2K)$ .
  - *Example:* A  $15 \times 15$  kernel drops from 225 to just 30 operations per pixel.

## The Cache Asymmetry Issue

- **Horizontal Pass:** Optimal spatial locality. Glides along adjacent columns mirroring row-major memory layout.
- **Vertical Pass:** Severely degraded. Reads vertically across different rows, overwhelming the cache and causing memory-bound slowdowns.

# Architecture: Producer-Consumer Pipeline

Solving the Fork-Join problem for compound operations

- **The Challenge:** 1D Opening requires 4 strict passes (H-Erosion  $\rightarrow$  V-Erosion  $\rightarrow$  H-Dilation  $\rightarrow$  V-Dilation). A standard Fork-Join doubles memory footprint and synchronization barriers.
- **The Pipeline Solution:**
  - Thread pool is halved within a single parallel region.
  - **Producers (Erosion):** Compute 1D passes and store in an intermediate buffer.
  - **Consumers (Dilation):** Fetch immediately from the buffer to compute final passes.
- **Result:** Overlaps execution, drastically reduces global memory traffic, and improves temporal locality.

# Experimental Setup



- **Environment:** Benchmarks conducted on a physical quad-core CPU architecture to strictly trace hardware scaling limits.
- **Dataset (BSDS500):** Diverse real-world image benchmark.
- **Scaling Methodology:**
  - *Strong Scaling:* Evaluating performance gains while increasing thread count on fixed image sizes.
  - *Weak Scaling:* Images upscaled by 2x and 4x to analyze throughput when proportional workload is added.

# Performance Results

○



UNIVERSITÀ  
DEGLI STUDI  
FIRENZE

Da un secolo, oltre.



# Algorithmic Reduction: 1D vs 2D Performance

○

- **The Baseline:** Comparison between 1D Separable Kernels and the standard 2D parallel baseline.
- **The Impact:** Transitioning to linear  $\mathcal{O}(2K)$  complexity acts as the primary performance multiplier.
- **Memory Bound:** As kernel size  $K$  increases, pure computations drop, but memory bandwidth becomes the new bottleneck (especially due to the vertical pass cache misses).

assets/figure6\_1d\_vs\_2d.png

# Architecture: Producer-Consumer Pipeline



- **Context:** Optimizing compound operations (Opening & Closing).
- **Pipeline Advantage:** Overlapping Erosion and Dilation inside a single parallel region avoids the strict Fork-Join barrier.
- **Performance Gain:** Yields up to **1.67x** speedup over the standard parallel baseline at low thread counts.
- **Scaling Behavior:** The pipeline

`assets/figure7_pipeline.png`

# Scalability Analysis: Strong & Weak Scaling

o

assets/figure3\_strong\_scaling.png

assets/figure4\_weak\_scaling.png

# Ineffective Strategies & Lessons Learned

What didn't work and why

- **Explicit SIMD Annotations:**

- `#pragma omp simd` proved ineffective in accelerating the core loops.
- This is likely due to the compiler's auto-vectorization already being optimal, or the `min/max` conditional branching limiting SIMD register exploitation.

- **Column-Major Traversal:**

- Attempting to traverse memory column-by-column to speed up the memory-bound Vertical 1D pass failed.
- RAM physical architecture is inherently *row-major*; software workarounds introduced unnecessary overhead without yielding any cache benefits.

## Peak Performance & Conclusions

o

### Optimal Configuration 22x Peak Speedup

Achieved combining 1D kernels and OpenMP ( $K = 15$ , 4 threads) against the sequential baseline.

- **Core Takeaway:** True acceleration requires a *multi-tiered* approach. Hardware scaling a flawed  $\mathcal{O}(K^2)$  algorithm is insufficient without algorithmic complexity reduction.
- **Future Developments:**
  - Extend the framework to Heterogeneous Computing (GPUs via **CUDA**).
  - The SIMT architecture and Shared Memory could definitively solve the vertical pass memory bottleneck.

# Visual Results (BSDS500)

Qualitative evaluation of compound operations

**Original (Grayscale)**



**Opening ( $K = 15$ )**



**Closing ( $K = 15$ )**



# Parallel Morphological Image Processing

*Thank you for listening!*  
*Any questions?*